NYC CLEAN TEAM INC.

Phase 1 Digital Service Platform

Customer operations and service workflow

Phase 1 Team Work Order

Implementation pathway, verification, and handover

Work order detail	Execution context
Prepared for	NYC Clean Team Inc.
Delivery team	AWTOMATIG product and engineering team
Delivery window	12 weeks from kickoff
Application baseline	Existing Next.js / React application, Node services, VPS, and MongoDB / Mongoose records

Purpose and delivery outcome

This work order directs the AWTOMATIG delivery team to extend NYC Clean Team Inc.'s existing application into a secure, database-backed commercial service workflow. The team will deliver customer intake and account features, quote review and approval, scheduling requests, administrative operations, payment and accounting integrations, transactional notifications, verification, deployment, and handover while retaining the current platform foundation.

The work is complete when the Phase 1 journeys pass the acceptance checks in this document, production deployment is handed over with the required technical records, and the agreed defect-support period is underway.

Delivery principles

- Extend the existing codebase, MongoDB / Mongoose database, authentication, and VPS configuration; do not rebuild or migrate the foundation.
- Keep workflow rules in modular services and APIs so presentation components do not own core business behavior.
- Use server-side authorization and ownership checks for every protected record and file operation.
- Treat MongoDB as the operational source of truth, Stripe as authoritative for transaction outcomes, and QuickBooks Online as the accounting system for synchronized records.

1. Existing platform and technical baseline

The reviewed baseline is a custom Next.js 16 and React 19 application with existing Node APIs and services, administrative components, authentication, MongoDB / Mongoose data, and deployment on the current VPS. The delivery team will inspect the current repository and production configuration at kickoff, identify reusable components and data structures, and record any implementation constraints before changing the application.

Layer	Phase 1 implementation direction
Web application	Extend the current Next.js / React routes, components, and responsive layouts.
Application services	Add or extend modular Node service and API boundaries for requests, quotes, status transitions, notifications, and integrations.
Operational data	Retain MongoDB / Mongoose records; add validated fields, indexed references, stable identifiers, and activity history as needed.
Files	Continue with the current storage approach where appropriate; associate validated attachments with service requests and enforce protected access.
Deployment	Use the current VPS and application deployment path; document environment configuration, release, and rollback steps.

2. Delivery workstreams

Workstream	Implementation outcomes
Customer accounts and properties	Registration, sign-in, recovery, company profile, customer ownership, and multiple service properties.
Service intake and scheduling requests	Service catalog, conditional questions, frequency selection, requested date/time, review before submission, secure attachments, and unique request IDs. Requested times remain requests until confirmed by staff.
Quotes and approval	Admin-created quotes, revisions, customer review, approval of the current revision, and request-linked history.
Administrative operations	Protected dashboard, request search and detail views, workflow status changes, scheduling confirmation, activity history, and payment status visibility.
Transaction workflow	One-time Stripe payment for an approved quote or other approved chargeable work; verified success/failure webhooks; recorded transaction and supported refund status.
Accounting synchronization	QuickBooks Online authorization and agreed customer, invoice, and payment-reference/status synchronization, with visible error state and authorized retry.
Notifications and documents	Transactional customer and admin email events; request-linked files with type/size validation and access controls.
Release and operational readiness	End-to-end QA, security checks, UAT, production deployment, source/repository access, technical documentation, and handover.

3. Twelve-week implementation pathway

The delivery sequence below is the working plan from kickoff. Workstreams may overlap where dependencies allow. The delivery lead will keep the scope and evidence aligned as the team moves through each checkpoint.

Window	Build focus	Checkpoint evidence
Week 1	Repository and infrastructure review; confirm current schemas, roles, service catalog, workflow states, integration mapping, backup configuration, and deployment path.	Architecture and data-flow notes; schema and entity map; confirmed workflow and integration decisions; access/dependency checklist.
Weeks 1–3	Extend MongoDB / Mongoose models, indexes, validation, request identifiers, role-aware APIs, activity events, and shared workflow services.	Reviewed schema changes; API/service boundary notes; tests for identifiers, ownership, and state transitions.
Weeks 3–6	Build customer accounts, company and property management, conditional service intake, review/submit flow, file association, and scheduling requests.	Working customer journey in UAT; responsive checks; persisted request/property relationships; file-access checks.
Weeks 5–8	Build quote creation and revisions, customer approval, protected admin dashboard, status/activity management, and scheduling confirmation.	Admin and customer quote journeys; current-revision approval evidence; status and history validation.
Weeks 7–10	Integrate Stripe, QuickBooks Online, and transactional email; implement webhook and synchronization error visibility and recovery actions.	Provider test evidence; success/failure paths; retry path; notification event verification.
Weeks 10–12	Run end-to-end QA and UAT, resolve in-scope defects, harden production configuration, deploy, and complete handover.	Completed acceptance matrix; release and rollback record; handover document set; production smoke-check evidence.

Checkpoint operating method

- At each checkpoint, the delivery lead records the build state, verification evidence, open defects, external dependencies, and next actions.
- The client reviewer validates the listed user journeys in the agreed UAT or production environment and reports reproducible failures against the acceptance matrix.
- The team fixes in-scope failures and records retest evidence before moving the affected journey to complete.

4. Architecture and integration requirements

Application boundaries

Keep page components focused on presentation and interaction. Put request transitions, quote revisions, authorization, attachment access, transaction handling, synchronization, and notification orchestration behind server-side service/API boundaries. Store stable cross-system identifiers on the related operational records and log meaningful state changes for administrators.

Stripe transaction flow

- Support one-time card payments and wallet methods only where enabled and supported by the provider, browser, device, and account configuration.
- Create payment attempts from an approved quote or other approved chargeable work and associate provider references with the relevant request.
- Verify webhook signatures before recording success or failure; use idempotent processing so repeated events do not duplicate state changes.
- Record transaction state and supported refund state for administrative review. Do not store full card numbers or CVC, and do not implement stored-card billing, subscriptions, or automatic recurring charges.

QuickBooks Online synchronization

- Confirm the synchronization direction, field mapping, and customer/invoice/payment references with the client before build.
- Authorize using QuickBooks OAuth and protect tokens in server-side secret configuration.
- Create or update the agreed customer record; create or synchronize an invoice for an approved quote or chargeable job; record the related payment reference/status after a successful Stripe transaction when supported by the agreed workflow and API.
- Persist synchronization state and error details for administrator review. Permit an authorized administrator to retry or complete the affected synchronization after correcting the cause.
- Do not implement automatic accounting reconciliation, payroll, or historical data migration.

Email and file workflows

Send event-based transactional messages for request receipt, quote availability and approval, transaction result, service confirmation and reminders, completion, and invoice availability where applicable. Send administrative alerts for new requests, quote approvals, transaction events, and cancellations. Keep messages tied to the underlying record and avoid placing secrets or unnecessary sensitive data in email.

Validate uploaded file type and size, associate file metadata with the service request, and enforce authorization on both metadata and file retrieval. Use time-limited access links when supported by the current storage mechanism. Direct API requests and altered record identifiers must not bypass ownership checks.

5. Security, data protection, and operational controls

Control area	Implementation and evidence
Authentication and roles	Retain the existing authentication foundation where appropriate. Enforce role checks server-side for administrative routes and protected actions.
Record ownership	Check the authenticated customer's ownership of company, property, request, quote, activity, and file records on every relevant server operation.
Input and session safety	Validate and normalize request data; apply appropriate request protections and rate limits; use secure session/token handling and password recovery.
Secrets and transport	Use HTTPS in production. Keep provider credentials and OAuth tokens in protected environment configuration; never place secrets in source control, client bundles, or handover examples.
Payment data	Use provider-hosted/tokenized payment flows. Do not persist full payment-card numbers or CVC in application records or logs.
Audit trail	Record actor, event, timestamp, and relevant request/quote state changes needed for operational history without logging secrets.
Backups and restoration	At kickoff, review current MongoDB and VPS/application backup configuration and document the available restoration procedure. Record observed configuration and the steps/owner for using the current procedure.

The team will report any material security or data-integrity issue discovered during delivery to the delivery lead and client contact, document its impact, and agree the corrective action before release.

6. Verification and acceptance matrix

Use this matrix as the completion checklist for the relevant workstream and for end-to-end UAT. Capture a concise test record with the environment, scenario, result, date, and linked defect or evidence where applicable.

Area	Pass condition
Accounts and ownership	A customer can register, sign in, recover access, and access only the customer's authorized company, property, request, quote, activity, and file records. Server APIs reject unauthorized and altered-ID access.
Multiple properties	A company can create and select at least two properties. Submitted service requests retain the correct company and property relationship.
Service intake	Conditional questions and required validation behave as configured. Review-before-submit works; submission creates one unique request ID and preserves requested date/time as a request.
Quote workflow	An administrator can create and revise a quote. The customer can review and approve the current revision; prior revisions and approval activity remain linked and visible.

Admin workflow	Authorized administrators can search, open, and update requests, confirm scheduling, view status and activity, and see transaction and synchronization status.
Stripe	A test transaction can complete and fail. Verified webhook events update the correct request exactly once. Supported refund status is recorded. No full card data or CVC is retained.
QuickBooks Online	Agreed customer and invoice records synchronize using the confirmed mapping. Payment reference/status is sent when supported. Failed sync is visible and an authorized retry succeeds after correction.
Notifications	Each configured customer and administrator event produces an accurate, correctly addressed message with no secrets or unrelated customer information.
Files and security	Valid files are linked to the correct request. Invalid type/size is rejected. Unauthorized users cannot access metadata or file content through UI or direct API routes.
Responsive and production	Representative customer and admin journeys work on mobile, tablet, and desktop layouts. Production smoke checks confirm secure transport, required configuration, protected admin access, and expected workflow behavior.
Backup procedure	The existing backup configuration and available restoration steps are reviewed and recorded at kickoff and included in the handover notes.
End-to-end journey	A representative request can be reviewed, quoted, revised, approved, processed through the applicable transaction and accounting paths, scheduled, and moved through completion with linked history.

7. Delivery documents and handover

Before closeout, prepare and review the following technical records. Store them with the project repository or the agreed secure handover location; do not place production secrets in documentation.

Handover item	Required content
Architecture overview	Application components, service/API boundaries, integration points, and current deployment topology.
Data model and workflow map	Entity relationships, schema extensions, roles, request/quote states, activity events, and key data flows.
Integration notes	Stripe event flow and webhook handling; QuickBooks mapping, sync direction, state/error handling, and retry steps; notification events.
Configuration guide	Required environment variable names, provider setup references, and safe configuration steps. Include no secret values.
Deployment and rollback guide	Build/release steps for the existing VPS, health checks, production smoke test, rollback procedure, and responsible account access path.
Security and file access notes	Authorization model, ownership checks, file validation and retrieval controls, and secret-handling expectations.
Backup/restoration notes	Observed current backup setup, available restoration procedure, and operational owner/contact to confirm at kickoff.

QA and UAT record	Acceptance matrix results, test environment, outstanding known issues, and retest evidence.
Admin quick-start	How authorized staff review requests, revise/send quotes, confirm scheduling, update status, inspect activity, and address integration errors.
Source and access handover	Complete Phase 1 source changes in the agreed repository, repository access for designated client users, and confirmation of administrative control for relevant service accounts and credentials.

8. Dependencies, boundaries, and support

Client and provider dependencies

- Provide timely repository, current VPS, database, and test-environment access through the agreed secure method.
- Confirm the service catalog, conditional intake questions, quote workflow, statuses, customer communication content, and QuickBooks field mapping during the foundation checkpoint.
- Provide authorized Stripe and QuickBooks Online test access and complete required provider OAuth/account approvals.
- Support domain/DNS verification required for production email delivery and identify authorized UAT reviewers.
- Review checkpoint evidence and report reproducible failures against the acceptance matrix so the team can resolve them within the delivery window.

Phase 1 boundary

Phase 1 extends the current customer and administrative platform. It establishes modular data and API boundaries that can support later staff permissions, assignments, and field workflows. Actual workforce assignment, dispatch, GPS tracking, field status, native mobile applications, automatic accounting reconciliation, payroll, historical migration, stored-card billing, subscriptions, and automatic recurring transactions are outside this work order.

Post-launch defect support

For 60 calendar days after production launch, the team will triage and correct reproducible defects where delivered functionality fails to conform to the accepted Phase 1 scope. Support covers diagnosis, in-scope correction, and release of the fix. New features, changed workflows, additional integrations, provider outages, and third-party configuration changes are not defects in delivered functionality. The team will document the reproduction steps, disposition, and resolution for each reported in-scope defect.

Completion record

The delivery lead closes the work order after the acceptance matrix and required handover records are complete, the production deployment and smoke check are documented, repository and account handover is confirmed, and the post-launch defect-support window has been communicated to the designated client contact.